

ISM3232 — Week 10 Lab

OOP I — Classes & Objects — Instructor Facilitation Guide

ISM3232 · Business Application Development · USF Muma College of Business

Module 10 · Unit 3 · Object-Oriented Design

Contents

1 Session Snapshot	2
2 Learning Objectives	2
3 Before Class — Setup Checklist	3
4 Materials Needed	3
5 Timing Plan (75 minutes)	3
6 Segment-by-Segment Walkthrough	4
6.1 Intro (0:00–0:04)	4
6.2 Part 1 — Write the Class (0:04–0:20, 16 min)	4
6.3 Part 2 — Create Instances (0:20–0:32, 12 min)	8
6.4 Part 3 — Verify Instance Independence (0:32–0:42, 10 min)	11
6.5 Part 4 — Seven pytest Tests (0:42–0:58, 16 min)	13
6.6 Part 5 — Ritual and Push (0:58–1:06, 8 min)	16
6.7 Stretch — Add a <code>summary()</code> Method (1:06–1:15, as time allows)	17
7 Wrap-Up (last ~9 minutes)	18
8 Appendix A — Full Answer Key (<code>models.py</code> + <code>main.py</code> + <code>tests/test_models.py</code>)	18
9 Appendix B — Extra Practice (only if the class finishes early)	21

1 Session Snapshot

Course	ISM3232 — Business Application Development
Session	Week 10 Lab — OOP I: Classes & Objects
Unit	Unit 3 · Object-Oriented Design
Class length	Full class period (75 minutes)
Format	Live code-along, ending with the full submission ritual
Prerequisites	Weeks 1–8: full Python foundations, functions, pytest, debugging discipline; Week 9 was the midterm
Student-facing lab page	Week 10 In-Class Lab — Module 6, “OOP I: Classes, Objects, Attributes, and Methods”
Parts covered	Part 1 (write the class) – Part 5 (ritual + push) + Stretch (summary() method)
Submission	GitHub URL in Canvas (no screenshots), completion credit

The midterm is behind the class; this is the first lab of Unit 3, and it opens a genuinely new paradigm — object-oriented programming. Everything before this unit used functions operating on plain dictionaries (`records = [{...}, {...}], def get_total(records): ...`); today, a `class` bundles data *and* the functions that operate on it into a single, reusable blueprint. The lab page’s own warning deserves emphasis: **no AI-generated classes** — every line must be explainable, and any AI-assisted portion needs to be pasted verbatim with an explanatory comment. This is a stricter standard than prior weeks’ general AI-use disclosure, worth stating explicitly at the start, since OOP syntax (especially `self` and `__init__`) is genuinely tempting to just paste from an AI tool without understanding.

2 Learning Objectives

By the end of this class period, students should be able to:

1. Define a class with `__init__`, and explain what `self` refers to and why every method needs it as its first parameter.
2. Create multiple independent instances of a class, and explain concretely why changing one instance’s state never affects another’s.
3. Write instance methods that read and modify an object’s own attributes.
4. Implement `__repr__` and explain how it changes what `print()` shows for an object.
5. Write `pytest` tests for a class, including a boundary case and a dedicated **instance-independence** test.

3 Before Class — Setup Checklist

- ☐ Rehearse explaining `self` at least two different ways before class — this is the single most common conceptual sticking point in this lab, and having more than one explanation ready (a physical analogy, a mechanical “Python passes the object automatically” explanation, and a live demonstration of calling a method both the normal way and the equivalent explicit way) covers more learning styles than any single explanation alone.
- ☐ Decide how strictly you’ll enforce “no AI-generated classes” today, and state it explicitly and specifically at the start — this is a stricter standard than a general AI-use disclosure; make clear what “must be able to explain every line” concretely means as a bar (e.g., could a student answer a cold-call question about any single line, unprompted).
- ☐ Have the “three independent instances” demonstration (Part 2) ready to pause on dramatically — this is the lab’s central payoff moment, and rushing past it undersells the entire point of the unit.

4 Materials Needed

- Terminal (zsh), VS Code, Python 3.10+
- Students: a fresh `~/ism3232/module06_oop/` project folder

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:04	Welcome: “bundling data and behavior together”	4
0:04–0:20	Part 1 — Write the class	16
0:20–0:32	Part 2 — Create instances	12
0:32–0:42	Part 3 — Verify instance independence	10
0:42–0:58	Part 4 — Seven pytest tests	16
0:58–1:06	Part 5 — Ritual and push	8
1:06–1:15	Stretch (<code>summary()</code> method) + wrap-up	9

Part 1 and Part 4 receive the most time — Part 1 because the new syntax (`class`, `__init__`, `self`) genuinely needs unhurried explanation, and Part 4 because “instance independence” is both a required test and a concept worth confirming lands solidly before students leave.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:04)

Say to the class:

“Every function you’ve written until now took data in as parameters and handed a result back — the data and the logic that worked on it lived in separate places. Today, a `class` bundles both together: the data (like a request’s amount and status) and the functions that work on that specific data (approve it, reject it, check if it needs review) live in one blueprint. Every object you create from that blueprint carries its own independent copy of the data. That independence — not the syntax — is the actual idea to walk away with today.”

Do: Write on the board: **class = blueprint, instance = one specific thing built from it**. Leave it visible through Part 2, where it becomes concrete.

6.2 Part 1 — Write the Class (0:04–0:20, 16 min)

Teaching goal: `class`, `__init__`, `self`, instance methods, and `__repr__` — five genuinely new pieces of syntax, each deserving individual explanation.

Say to the class:

“One class, five new pieces of syntax. I’m going to slow down for every single one, because OOP syntax looks deceptively simple and is very easy to type without actually understanding — which is exactly what today’s ‘no AI-generated classes’ rule is designed to prevent.”

Live-code this:

```
cd ~/ism3232/module06_oop
python3 -m venv .venv && source .venv/bin/activate
pip install pytest ruff && pip freeze > requirements.txt
echo '.venv/' > .gitignore && echo '__pycache__/' >> .gitignore
touch models.py main.py && mkdir -p tests && touch tests/__init__.py tests/test_models.py
code models.py
```

```
# models.py
# Author: [Your Name]

class BusinessRequest:
    """Represents a single business purchase or travel request."""

    def __init__(self, request_id, requester, category, amount):
```

```

        self.request_id = request_id
        self.requester = requester
        self.category = category
        self.amount = amount
        self.status = 'Pending' # default

    def approve(self):
        """Mark this request as approved."""
        self.status = 'Approved'

    def reject(self):
        """Mark this request as rejected."""
        self.status = 'Rejected'

    def requires_review(self):
        """Return True if amount exceeds 1000."""
        return self.amount > 1000

    def __repr__(self):
        return f'BusinessRequest({self.request_id}, {self.requester}, ${self.amount}, {self.

```

Line-by-line explanation:

- `class BusinessRequest:` — class begins a class definition, exactly like `def` begins a function — say explicitly, this is a **blueprint**, not yet a real object; nothing exists in memory as an actual “business request” until Part 2 creates one *from* this blueprint.
- `"""Represents a single business purchase or travel request."""` — a docstring for the *class itself*, describing what any object built from this blueprint represents — the same docstring convention from Week 7’s functions, now at the class level.
- `def __init__(self, request_id, requester, category, amount):` — **the constructor**, called automatically the moment a new object is created from this class. Say explicitly, precisely: `__init__` is not called directly by name anywhere in this lab — it runs automatically, once, whenever `BusinessRequest(...)` is called (Part 2 does this three times). The double underscores before and after `init` mark it as a **special method** Python itself recognizes and calls at a specific moment — not a naming convention a developer invents.
- `self` — **this is the single most important word in this entire lab, and worth explaining more than once, in more than one way.** Say it plainly first: `self` refers to *this specific object being created or acted on* — when Part 2 creates `req_101 = BusinessRequest(101, 'Taylor', 'Travel', 1200)`, inside `__init__`, `self` *is* `req_101`, even though `req_101` doesn’t have that name yet at the moment `__init__` is running. Every method in this class takes `self` as its first parameter — not because a developer chose to, but because that’s how Python passes “which specific object is this method being called on” into every method automatically. **A genuinely useful mechanical framing:** `req_101.approve()` is secretly

equivalent to `BusinessRequest.approve(req_101)` — Python quietly passes `req_101` in as `self` for you; the dot-notation call just hides that first argument. Consider writing both forms on the board side by side.

- `self.request_id = request_id` (and the three lines below it) — each line takes a plain parameter (`request_id`, holding whatever number was passed in) and stores it as an **attribute** of *this specific object*, accessible later as `self.request_id` from inside any method, or `req_101.request_id` from outside the class entirely. Say explicitly: **the parameter `request_id` and the attribute `self.request_id` are not the same thing, even though they share a name** — the parameter exists only briefly, while `__init__` is running; the attribute persists on the object for as long as the object itself exists.
- `self.status = 'Pending'` # default — note this line does **not** take a parameter — every new `BusinessRequest`, regardless of what's passed to `__init__`, starts with `status` set to `'Pending'`, unconditionally. This directly answers one of Part 3's required reflection questions ("where does the default status come from") — worth flagging that connection now.
- `def approve(self):` — an **instance method**: still takes `self` as its first parameter (every method inside a class does), but takes no *other* parameters here, since approving a request doesn't need any additional information beyond "which request." `self.status = 'Approved'` — modifies *this specific object's* `status` attribute, permanently, for as long as the object exists.
- `def requires_review(self):` — return `self.amount > 1000` — reads (rather than modifies) `self's` own `amount` attribute — the exact same comparison logic as Week 7's standalone `requires_review()` function, now living *inside* the object it's checking, reading the object's own data directly via `self` instead of receiving it as a separate parameter.
- `def __repr__(self):` — another special method (double underscores again), this one controlling **what `print()` shows** when given an object of this class. Without a custom `__repr__`, printing a `BusinessRequest` object would show something unhelpful like `<models.BusinessRequest object at 0x...>` — a memory address, not useful information. This `__repr__` instead builds a readable string using the object's own current attribute values.

Common student mistakes to watch for:

- Forgetting `self` as the first parameter in a method definition — this doesn't error immediately, but calling the method later fails in a confusing way (`TypeError: approve() takes 0 positional arguments but 1 was given`, since Python still tries to pass the object in automatically); worth demonstrating live once if it comes up naturally, since it's a very common early mistake.
- Confusing `self.amount` (an attribute, read inside a method) with a bare `amount` (which would refer to a completely separate, undefined local variable inside most of these methods, since none of them except `__init__` actually receive `amount` as a parameter) — a good moment to point at `requires_review's` single line and ask "why does this say `self.amount` and not just `amount`?"
- Writing `def __init__(self):` with no other parameters, then trying to hardcode values in-

stead of accepting them — technically avoids the parameter-passing confusion, but produces a class where every instance would be identical, defeating Part 2's entire point of creating three genuinely different requests.

Check for understanding: “If I removed `self` entirely from `requires_review`'s parameter list, and called `req_101.requires_review()`, what would happen?” (TypeError, complaining about too many arguments — Python still automatically passes `req_101` in as an argument regardless of whether the method's definition expects it; removing `self` from the definition doesn't stop Python from trying to pass it, it just means there's no parameter slot to receive it.)

6.3 Part 2 — Create Instances (0:20–0:32, 12 min)

Teaching goal: Create three genuinely independent objects from the same class, call methods on them, and witness — concretely, not just by assertion — that changing one never affects the others.

Say to the class:

“Three requests, from the same blueprint, about to become three genuinely separate things in memory. Watch closely when we approve one and reject another — the third stays completely untouched. This is the payoff of everything in Part 1.”

Live-code this:

```
# main.py
from models import BusinessRequest

req_101 = BusinessRequest(101, 'Taylor', 'Travel', 1200)
req_102 = BusinessRequest(102, 'Jordan', 'Equipment', 450)
req_103 = BusinessRequest(103, 'Morgan', 'Software', 3500)

# Check initial state
print(req_101)
print(req_102)
print(req_103)

# Call methods
print(req_101.requires_review()) # True
print(req_102.requires_review()) # False

req_101.approve()
req_102.reject()

# Verify independence
print(req_101.status) # Approved
print(req_102.status) # Rejected
print(req_103.status) # Pending -- unchanged

# Repr
print(req_101)
print(req_102)
```

Line-by-line explanation:

- `from models import BusinessRequest` — importing the class itself (not a function) from the local module, same import mechanics as Week 7's function imports.
- `req_101 = BusinessRequest(101, 'Taylor', 'Travel', 1200)` — this is where `__init__` actually runs, once, right now — `self` inside that run of `__init__` is whatever `req_101` ends up referring to. Say explicitly: **three separate calls to `BusinessRequest(...)` create three separate objects, each with its own independent set of attributes**, even though they were all built from the exact same blueprint.
- `print(req_101)` (before any methods are called) — this is where `__repr__` fires automatically: `print()` on any object calls `__repr__` (or a closely related method, `__str__`, not covered today) to decide what to display — worth stating explicitly that this isn't `print()` doing anything special for this particular class; it's a general Python behavior that this class's custom `__repr__` is customizing.
- `req_101.requires_review()`, `req_102.requires_review()` — same method, two different objects, two different results (`True` for the \$1,200 Travel request, `False` for the \$450 Equipment one) — because each call's `self` is a genuinely different object with its own amount.
- `req_101.approve()`, `req_102.reject()` — **this is the moment worth pausing on dramatically**. Neither of these calls touches `req_103` at all — there's no code anywhere that could cause them to.
- The three status-printing lines afterward — **the concrete proof of independence**: `req_101.status` is now `'Approved'`, `req_102.status` is `'Rejected'`, and `req_103.status` is still `'Pending'`, completely unaffected by anything that happened to the other two.

Run it. Verified output:

```
BusinessRequest(101, Taylor, $1200, Pending)
BusinessRequest(102, Jordan, $450, Pending)
BusinessRequest(103, Morgan, $3500, Pending)
True
False
Approved
Rejected
Pending
BusinessRequest(101, Taylor, $1200, Approved)
BusinessRequest(102, Jordan, $450, Rejected)
```

Say explicitly, pointing at the final two lines specifically: “Notice `__repr__` picks up the *current* status automatically — `req_101`'s printed representation now says `Approved`, not `Pending`, even though we never touched `__repr__` itself. It reads `self.status` fresh, every time it's called, which is exactly why it correctly reflects whatever state the object is *currently* in.”

Common student mistakes to watch for:

- Expecting `req_103.status` to have changed somehow, from a vague sense that “something happened to the requests” — this is worth taking seriously as a genuine point of early OOP confusion, not dismissing; have the student trace through the code line by line and identify

any line that references req_103 between its creation and the final print (there isn't one) as the concrete proof.

- Confusing the three variable names (req_101, req_102, req_103) with the request_id attribute values (101, 102, 103) — these happen to numerically correspond in this example, which is a deliberate readability choice, not a required rule; a good moment to note that req_101 is just a Python variable name (could be called anything), while 101 is data stored inside the object.

Check for understanding: “If I added a fourth line, req_104 = req_101, and then called req_104.approve(), would req_101.status also become 'Approved'?” (Yes — this is worth flagging as a genuinely important, different scenario from today's three independent objects: req_104 = req_101 doesn't create a new object at all, it just gives a *second name* to the *same* existing object; req_104.approve() and req_101.approve() would be indistinguishable, since they're both acting on the identical object in memory. Contrast this explicitly with BusinessRequest(101, ...) called a *second* time, which *would* create a genuinely new, independent object even with identical data — worth stating plainly if a curious student asks, though not required depth for this specific lab's core content.)

6.4 Part 3 — Verify Instance Independence (0:32–0:42, 10 min)

Teaching goal: A required, written reflection — five questions, answered as a comment block — confirming Part 2’s demonstration genuinely translated into conceptual understanding, not just observed output.

Say to the class:

“Five questions, answered in your own words, as a comment block at the top of `main.py`. I want genuine answers, not restated code — if you can’t answer one confidently, that’s exactly the thing to ask about right now.”

Add this comment block, and have every student answer it individually before any discussion:

```
# --- OOP Lab Questions ---  
# 1. After calling req_101.approve(), what is req_102.status? ____  
# 2. Why doesn't approving req_101 affect req_102? ____  
# 3. What does 'self' refer to inside the approve() method? ____  
# 4. What is the type of req_101? ____ (hint: use type())  
# 5. Where does the default status 'Pending' come from? ____
```

Model strong answers explicitly, since these are worth confirming precisely, not just approximately:

1. **'Pending'** (unchanged) — a direct, checkable fact from Part 2’s actual run.
2. **Because req_101 and req_102 are separate objects, each with its own independent copy of every attribute, including status** — a method call on one object only ever touches that specific object’s own data.
3. **req_101 itself**, at the moment `req_101.approve()` is called — `self` is however Python’s automatic argument-passing resolves for *this specific call*; a different call, `req_102.approve()`, would have `self` refer to `req_102` instead, inside that separate invocation of the same method code.
4. **<class 'models.BusinessRequest'>** — have students actually run `print(type(req_101))` to confirm, rather than guess at the exact formatting.
5. **`__init__`’s unconditional `self.status = 'Pending'` line** — every object gets this value automatically on creation, regardless of what’s passed into `__init__`, since `status` is never one of `__init__`’s parameters.

Common student mistakes to watch for:

- Answering question 2 with a description of *what* happens (restating “req_102 stays Pending”) rather than *why* — redirect explicitly toward the independence concept itself, not just the observed fact.
- Answering question 3 vaguely (“self refers to the object”) without connecting it to *which spe-*

cific call — push for the more precise “self refers to req_101, specifically because this line is req_101.approve()” framing.

Check for understanding: This entire part *is* the check for understanding — treat student answers to these five questions, read individually as you circulate, as your primary signal for whether Part 1–2’s concepts landed, and address gaps directly and immediately rather than waiting for Part 4’s tests to surface them indirectly.

6.5 Part 4 — Seven pytest Tests (0:42–0:58, 16 min)

Teaching goal: Seven tests, including Week 7’s now-familiar boundary case, plus a **new kind of test unique to OOP**: a dedicated independence test, directly encoding Part 2–3’s central lesson as executable, permanent verification.

Say to the class:

“Seven tests. Six of them check individual behaviors — status changes, review thresholds, the boundary case exactly like Week 7. The seventh is new: a test whose entire job is proving instance independence, in code, so it’s never just something we observed once and trusted.”

Live-code this:

```
# tests/test_models.py
from models import BusinessRequest

def test_default_status_is_pending():
    req = BusinessRequest(1, 'Test', 'Travel', 500)
    assert req.status == 'Pending'

def test_approve_changes_status():
    req = BusinessRequest(2, 'Test', 'Travel', 500)
    req.approve()
    assert req.status == 'Approved'

def test_reject_changes_status():
    req = BusinessRequest(3, 'Test', 'Travel', 500)
    req.reject()
    assert req.status == 'Rejected'

def test_requires_review_over_limit():
    req = BusinessRequest(4, 'Test', 'Travel', 1500)
    assert req.requires_review() is True

def test_requires_review_under_limit():
    req = BusinessRequest(5, 'Test', 'Travel', 500)
    assert req.requires_review() is False

def test_requires_review_at_boundary():
    req = BusinessRequest(6, 'Test', 'Travel', 1000)
    assert req.requires_review() is False # > not >=
```

```
def test_instances_are_independent():
    req1 = BusinessRequest(7, 'A', 'Travel', 500)
    req2 = BusinessRequest(8, 'B', 'Travel', 500)
    req1.approve()
    assert req2.status == 'Pending'
```

Line-by-line explanation:

- The first three tests — `test_default_status_is_pending`, `test_approve_changes_status`, `test_reject_changes_status` — each creates its **own fresh instance**, worth explicitly noting: every test builds a brand-new `BusinessRequest`, rather than sharing one object across tests, since a shared object's state from one test could otherwise leak into and corrupt the next.
- `test_requires_review_at_boundary` — Week 7's exact boundary-testing discipline, now applied to a method instead of a standalone function — same principle, same reason: confirms > 1000 , not ≥ 1000 , at the exact threshold.
- `test_instances_are_independent` — **this is the lab's new, central test**. Read it slowly: two separate objects are created (`req1`, `req2`), only `req1` is approved, and the assertion checks `req2.status` — specifically confirming the *other* object was never touched. Say explicitly: **this test doesn't just check that `approve()` works — it checks that `approve()`'s effect is correctly scoped to exactly one object**, which is a fundamentally different, and in some ways more important, property to verify than any single method's individual correctness.

Run it:

```
pytest -v
```

Verified output — all seven pass:

```
tests/test_models.py::test_default_status_is_pending PASSED
tests/test_models.py::test_approve_changes_status PASSED
tests/test_models.py::test_reject_changes_status PASSED
tests/test_models.py::test_requires_review_over_limit PASSED
tests/test_models.py::test_requires_review_under_limit PASSED
tests/test_models.py::test_requires_review_at_boundary PASSED
tests/test_models.py::test_instances_are_independent PASSED
7 passed
```

A genuinely worthwhile live demo, if time allows: temporarily break independence on purpose — change `__init__`'s `self.status = 'Pending'` to a class-level assignment outside `__init__` (a subtle, real Python bug pattern involving mutable class attributes, more advanced than this lab requires in full depth, but worth a simplified version if you want to show `test_instances_are_independent` actually catching something). If this feels like too large a detour, it's completely fine to skip — the test passing correctly as written is itself sufficient

demonstration for this lab's scope.

Common student mistakes to watch for:

- Writing `test_instances_are_independent` using the **same** object for both actions (a copy-paste slip creating `req1` twice instead of `req1` and `req2`) — this would make the test tautological (of course an object's status matches itself) rather than actually testing independence; a good “does this test genuinely test what its name claims” check.
- Treating all seven tests as equally routine, without recognizing `test_instances_are_independent` as conceptually different from the other six — worth explicitly asking a student to categorize the seven tests into groups (state-changing behavior, boundary condition, independence) to confirm the distinction registered.

Check for understanding: “Could you write a test proving `req_103` from Part 2 was never touched by anything that happened to `req_101` or `req_102`, using this same pattern?” (Yes — create three instances, act on two of them, assert the third's state is unchanged — a direct generalization of `test_instances_are_independent` to three objects instead of two, confirming the pattern, not just this one specific instance of it, is understood.)

6.6 Part 5 — Ritual and Push (0:58–1:06, 8 min)

Teaching goal: The established ritual, on genuinely new (OOP) content — confirming the habit transfers cleanly to a new unit, not just the unit it was originally taught in.

Say to the class:

“Same ritual, new kind of code entirely. If this feels routine by now, that’s the habit working exactly as intended.”

Live-code this:

```
ruff format . && ruff check . && pytest -v  
git add . && git commit -m 'lab 10: OOP I BusinessRequest class' && git push
```

Common student mistakes to watch for: None new this week — pure repetition; the main thing worth confirming is that the ritual runs without needing the steps spelled out, four units of new material after it was first introduced.

Check for understanding: “Commit message must include ‘lab 10’ — quick, what’s yours?” (A fast, low-stakes final check before the push, same pattern as every prior week.)

6.7 Stretch — Add a `summary()` Method (1:06–1:15, as time allows)

Frame as a genuinely good closer if the room reaches it — real practice adding a new method to an existing, working class:

```
def summary(self):  
    """Return a one-line, human-readable summary of this request."""  
    return f'Request {self.request_id} by {self.requester} | {self.category} | ${self.amount}'
```

Verified output (called on `req_101`, after `approve()`):

Request 101 by Taylor | Travel | \$1,200.00 | Approved

And a test for it, in `tests/test_models.py`:

```
def test_summary_includes_key_fields():  
    req = BusinessRequest(101, 'Taylor', 'Travel', 1200)  
    req.approve()  
    result = req.summary()  
    assert 'Taylor' in result  
    assert 'Approved' in result  
    assert '1,200.00' in result
```

One thing worth naming if you demo this: `summary()` reads `self.request_id`, `self.requester`, `self.category`, `self.amount`, and `self.status` — **all five of the object's attributes at once** — worth pointing out that this is a genuinely natural use for a method: pulling together everything an object knows about itself into one readable output, without needing any information from outside the object (no parameters beyond `self`).

7 Wrap-Up (last ~9 minutes)

Review the submission checklist together:

- ☐ Git commit made, with a message including “lab 10”
- ☐ `models.py` contains `BusinessRequest` with `__init__`, four methods, and `__repr__`
- ☐ `main.py` creates three instances, calls methods, and answers all five reflection questions in a comment block
- ☐ `tests/test_models.py` contains all seven tests, including the boundary case and the independence test, all passing
- ☐ Full ritual run (format, lint, test, commit, push)
- ☐ GitHub repository URL pasted into Canvas

Preview Week 11: “Today, one class stood alone. Next week: composition (one class containing objects of another class) and inheritance (one class building on another) — the two primary ways classes relate to each other in real object-oriented systems.”

8 Appendix A — Full Answer Key (`models.py` + `main.py` + `tests/test_models.py`)

```
# models.py
# Author: [Your Name]

class BusinessRequest:
    """Represents a single business purchase or travel request."""

    def __init__(self, request_id, requester, category, amount):
        self.request_id = request_id
        self.requester = requester
        self.category = category
        self.amount = amount
        self.status = 'Pending' # default

    def approve(self):
        """Mark this request as approved."""
        self.status = 'Approved'

    def reject(self):
        """Mark this request as rejected."""
        self.status = 'Rejected'

    def requires_review(self):
```

```

        """Return True if amount exceeds 1000."""
        return self.amount > 1000

    def __repr__(self):
        return f'BusinessRequest({self.request_id}, {self.requester}, ${self.amount}, {self.status})'

```

```

# main.py
# --- OOP Lab Questions ---
# 1. After calling req_101.approve(), what is req_102.status? Pending
# 2. Why doesn't approving req_101 affect req_102? They are separate
#    objects, each with its own independent copy of every attribute.
# 3. What does 'self' refer to inside the approve() method? Whichever
#    specific object the method was called on -- req_101 in this case.
# 4. What is the type of req_101? <class 'models.BusinessRequest'>
# 5. Where does the default status 'Pending' come from? __init__'s
#    unconditional self.status = 'Pending' line.

```

```

from models import BusinessRequest

req_101 = BusinessRequest(101, 'Taylor', 'Travel', 1200)
req_102 = BusinessRequest(102, 'Jordan', 'Equipment', 450)
req_103 = BusinessRequest(103, 'Morgan', 'Software', 3500)

print(req_101)
print(req_102)
print(req_103)

print(req_101.requires_review())
print(req_102.requires_review())

req_101.approve()
req_102.reject()

print(req_101.status)
print(req_102.status)
print(req_103.status)

print(req_101)
print(req_102)

```

```

# tests/test_models.py
from models import BusinessRequest

def test_default_status_is_pending():
    req = BusinessRequest(1, 'Test', 'Travel', 500)
    assert req.status == 'Pending'

def test_approve_changes_status():
    req = BusinessRequest(2, 'Test', 'Travel', 500)
    req.approve()
    assert req.status == 'Approved'

def test_reject_changes_status():
    req = BusinessRequest(3, 'Test', 'Travel', 500)
    req.reject()
    assert req.status == 'Rejected'

def test_requires_review_over_limit():
    req = BusinessRequest(4, 'Test', 'Travel', 1500)
    assert req.requires_review() is True

def test_requires_review_under_limit():
    req = BusinessRequest(5, 'Test', 'Travel', 500)
    assert req.requires_review() is False

def test_requires_review_at_boundary():
    req = BusinessRequest(6, 'Test', 'Travel', 1000)
    assert req.requires_review() is False # > not >=

def test_instances_are_independent():
    req1 = BusinessRequest(7, 'A', 'Travel', 500)
    req2 = BusinessRequest(8, 'B', 'Travel', 500)
    req1.approve()
    assert req2.status == 'Pending'

```

Stretch (summary() method):

```

def summary(self):
    """Return a one-line, human-readable summary of this request."""
    return f'Request {self.request_id} by {self.requester} | {self.category} | ${self.amount}

```

9 Appendix B — Extra Practice (only if the class finishes early)

Four required parts plus the ritual fill the full class period at a normal pace, especially given Parts 1–3’s deliberate conceptual depth. If a section moves unusually fast:

Extra — a second independence test, three objects. Have students write `test_three_instances_independent` creating three `BusinessRequest` objects, approving one, rejecting a second, and asserting the third remains 'Pending' — a direct generalization of Part 4’s two-object independence test to three, per this guide’s own check-for-understanding question.

Extra — a `reset()` method. Have students add `def reset(self): self.status = 'Pending'` to `BusinessRequest`, write a test confirming it correctly returns an approved or rejected request back to 'Pending', and discuss briefly: is a `reset()` method a realistic feature for a real business-request system, or mostly a testing convenience? (No single right answer — a good discussion prompt connecting today’s OOP mechanics back to genuine business-system design judgment.)